LangGraph & LangChain

Nov 6, 2026 · @Ashutosh

Summary

LangGraph is the low-level runtime that runs agents as stateful graphs, and LangChain is the high-level agent API built on top of it. Since both reached v1.0 in October 2025, the recommended path is to start with LangChain's `create_agent` and drop down to LangGraph when a workflow needs custom branching, long-running durability or human approval steps.

1. LangChain and LangGraph

LangGraph is the foundation and LangChain sits on top of it, not the other way round. Both reached v1.0 on 22 October 2025, with a commitment to no breaking changes until 2.0 (LangChain blog).

LangChain

LangChain is the fast path to a working agent. It gives a standard interface to hundreds of model providers, so the model can be swapped without rewriting the app. Version 1.0 narrowed the package to three things:

- **`create_agent`**: one function that builds the standard agent loop from a model, a list of tools and a system prompt. It replaces the older `create_react_agent` from `langgraph.prebuilt`.
- **Middleware**: hooks into each step of the loop. Built-in middleware covers human approval of tool calls, summarising long histories, and redacting personal data.
- **Standard content blocks**: one format for model output (text, reasoning, citations, tool calls) across providers.

Older chains and utilities moved to a separate `langchain-classic` package.

```
from langchain.agents import create_agent

agent = create_agent(
    model="openai:gpt-5",
    tools=[get_weather],
    system_prompt="Help the user by fetching the weather in their city.",
)
result = agent.invoke({"messages": [{"role": "user", "content": "Weather in
Kochi?"}]})
```

LangGraph

LangGraph models an agent or workflow as a graph. Three primitives do the work:

- **State**: a typed object shared by every step, holding messages and any other working data.
- **Nodes**: functions that read the state and return updates, such as a model call, a tool run or plain code.
- **Edges**: the connections between nodes. Conditional edges branch on the state, and edges can loop back, which a simple chain cannot do.

On top of these, LangGraph supplies the runtime features production agents need. **Checkpointers** save state after every step, so a run survives a server restart and can resume later. **Interrupts** pause a run for a human to approve or edit, then continue. Memory comes in two kinds: short-term memory lives in the thread's state, and long-term memory lives in a store that persists across sessions.

Which to use

	LangChain 1.0	LangGraph 1.0
Level	High-level agent API	Low-level graph runtime
Core building block	`create_agent` + middleware	State, nodes, edges
Control flow	Fixed loop: model → tools → answer	Any graph: branches, loops, parallel steps

	LangChain 1.0	LangGraph 1.0
Best for	Standard tool-using agents, shipping fast	Custom workflows, long-running jobs, human approval, tight cost or latency control
Relationship	Runs on LangGraph	Can embed `create_agent` agents as nodes

The usual advice is to start with LangChain and drop down to LangGraph when the default loop no longer fits. Because a LangChain agent is itself a LangGraph graph, it can be placed inside a larger custom graph without a rewrite. The wider ecosystem adds **LangSmith** for tracing, evaluation and deployment, and **Deep Agents** for long-running agents that plan and use sub-agents.

2. A custom LangGraph example

The sketch below builds by hand the loop that `create_agent` gives for free, then adds what only a custom graph allows: a checkpointer for durability and a pause before a sensitive tool runs. It is illustrative; exact imports can differ by version, so check the current docs.

```
from typing import Annotated, TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode
from langgraph.checkpoint.memory import InMemorySaver
from langchain.chat_models import init_chat_model

class State(TypedDict):
    messages: Annotated[list, add_messages]   # new messages are appended

tools = [search_orders, issue_refund]
model = init_chat_model("anthropic:claude-sonnet-4-5").bind_tools(tools)

def call_model(state: State):
    return {"messages": [model.invoke(state["messages"])]}

def route(state: State):
    last = state["messages"][-1]
    return "tools" if last.tool_calls else END
```

```
graph = StateGraph(State)
graph.add_node("model", call_model)
graph.add_node("tools", ToolNode(tools))
graph.add_edge(START, "model")
graph.add_conditional_edges("model", route)   # branch: tools or finish
graph.add_edge("tools", "model")             # loop back with tool results

app = graph.compile(
    checkpointer=InMemorySaver(),   # use a Postgres/SQLite saver in
production
    interrupt_before=["tools"],     # pause for human approval before any
tool runs
)

config = {"configurable": {"thread_id": "customer-42"}}
app.invoke({"messages": [("user", "Refund my last order")]}, config)
# ...a human reviews the pending tool call, then resumes the same thread:
app.invoke(None, config)
```

What each piece does:

- **`State`** holds the conversation; the `add_messages` reducer appends rather than overwrites.
- **Conditional edge** `route` is the decision point: a tool call loops to the tools node, a plain answer ends the run.
- **Checkpointer +** **`thread_id`** save state after every step, so the thread can be resumed after a pause or a crash.
- **`interrupt_before`** stops the run before tools execute. Finer-grained control is possible with the `interrupt()` function inside a node, which pauses only for specific actions such as refunds.

The same result for a standard agent takes a few lines with `create_agent` plus the human-in-the-loop middleware. The custom graph earns its extra code when the workflow needs steps the default loop does not have, such as a fixed validation step, parallel branches or several agents.

Sources

- LangChain and LangGraph Agent Frameworks Reach v1.0 Milestones, LangChain blog, 22 October 2025

- LangChain & LangGraph 1.0 alpha releases, LangChain blog, 2 September 2025
- LangChain documentation